

Alexandria: Statistical Text Analysis Tool Project Plan

1. Project Overview

1.1 Project Title

Alexandria: The statistical text analysis tool.

1.2 Problem Summary

Many kinds of text-based research, be it linguistics or politics e. g., require reliable, algorithmic, repeatable statistical analysis (non-LLM-based). Generative AI is unreliable for precise quantitative data analysis, like word counts and exact appearances in text. Researchers need a deterministic tool to ensure reliability and truthfulness of data.

1.3 Intended Audience

Librarians, researchers, linguists, teachers, and students who work with large texts or text collections and need accurate text search, quantitative analysis, text comparison, and tools for saving research results and quotations.

1.4 Main Features & Components

1. Search & Extraction: Find words and phrases (including their variations) and show them in context with their location in the text.
2. Statistical Analysis: Calculate exact word frequencies, showing how often each word or phrase is used in the text.
3. Multi-Text Comparison: Compare the difference in the usage of words and phrases across multiple documents at the same time.
4. Data saving: Save books/texts, search results, statistics and quotations to a database and user profile.

2. Project Objectives

- To develop a working JavaFX desktop application for quantitative text analysis within 8 weeks.
- To allow users to open and view text files directly in the application.
- To provide fuzzy search for words and phrases, including different forms of the same word.

- To show search results together with their surrounding text and location when available.
- To calculate text statistics, including the most frequently used words and the number of occurrences of a selected word or phrase.
- To exclude stop words from text statistics.
- To allow users to compare two or more texts using word usage statistics.
- To provide a basic user profile for storing personal information, books/texts, search results, text statistics, and quotations.
- To allow users to save and view statistical results and quotations.
- To develop and test the application using the development, testing, version control, and automation practices used in the course.

3. Scope and Deliverables

3.1 Project Scope

Alexandria is a JavaFX-based desktop application for quantitative analysis of texts and text sets. The application allows users to open local text files, search for words and phrases, calculate text statistics, and compare results between several texts.

The system also provides a user profile where users can store personal information and access saved statistical results and quotations.

The project focuses on providing accurate and repeatable quantitative results that can be checked against the original text.

3.2 What the project includes

- JavaFX desktop application,
- Text input and TXT/PDF support,
- Text search and fuzzy search,
- Text statistics and stop word filtering,
- Multi-text comparison,
- User profile,
- Saving books/texts, search results, statistics, and quotations,
- Automated testing,

- CI pipeline,
- Git and Trello for project management.

3.3 Optional Features

If there is enough time, the project may also include:

- Text tokenization for additional text processing
- Text vectorization
- Additional analysis based on vectorized text data
- Similarity analysis between texts based on their vector representations
- Support for additional text file formats

3.4 What the project does not include

- A web or mobile version of the application
- Generative AI or LLM-based text interpretation
- Automatic interpretation of the meaning or content of texts
- Automatic generation of research conclusions
- Online collaboration between users

3.5 Project Deliverables

- Product vision and project planning documentation
- User stories and prioritized product backlog
- User interface design
- Application architecture and data design
- A working JavaFX desktop application
- Text upload and viewing functionality
- Text search functionality
- Text statistics functionality
- Text comparison functionality
- A basic user profile
- Saved books/texts, search results, statistical results, and quotations

- Automated tests and test results
- CI pipeline for automated build and test execution
- Final project documentation
- A working project demonstration

4. Project Timeline

The project lasts eight weeks and has four sprints.

4.1 Sprint 1

17.08-30.08. Project planning and setup.

Planned work:

- Set up the Java project and GitHub repository.
- Set up Trello and the product backlog.
- Create the Product Vision.
- Create and prioritize user stories.
- Estimate user stories with Story Points.
- Prepare the Project Plan.
- Research PDF text extraction.
- Create the first PDF parser.

Milestones:

- Product Vision completed.
- Product backlog completed.
- Project Plan completed.
- PDF parser created.
- Sprint 1 review: 31.08.

Dependencies:

- Project planning before development.

4.2 Sprint 2

31.08-13.09. Core functionality and first tests.

Planned work:

- Text input and TXT/PDF upload
- Text viewing
- Search and fuzzy search
- Text statistics
- Basic user profile
- Unit and integration tests
- Code coverage and clean code

Milestones:

- Text input, search, and statistics working
- Tests added
- Sprint 2 review: 14.09

Dependencies:

- Text loading and parsing before search and statistics.

4.3 Sprint 3

14.09-27.09. Extended features and automation.

Planned work:

- Add comparison of multiple texts.
- Compare words and phrases between texts.
- Save and view search results.
- Save and view statistics.
- Save and view quotations.
- Configure Jenkins.
- Create a build and test pipeline.
- Start working with Docker.

Milestones:

- Text comparison working.

- Saved statistics and quotations available.
- Jenkins pipeline working.
- Docker setup started.
- Sprint 3 review: 28.09.

Dependencies:

- Search and statistics before text comparison.
- User profile and database before saving results.

4.4 Sprint 4

28.09-05.10. MVP, testing, and presentation.

Planned work:

- Fix bugs.
- Complete automated tests.
- Continue working with Docker.
- Work with Kubernetes and SecDevOps.
- Complete project documentation.
- Prepare the demo and presentation.

Optional work:

- Text tokenization.
- Text vectorization.
- Analysis based on vectorized text.
- Text similarity analysis.
- Support for more text file formats.

Milestones:

- MVP completed.
- Final testing completed.
- Documentation completed.
- Presentation prepared.

Dependencies:

- Main features before MVP.
- Stable build before Docker and Kubernetes.

5. Resource Allocation

5.1 Team Members and Roles

Member / Role	Responsibilities
Veikka (Backend Developer)	Responsible for the core analysis engine. Veikka implements the deterministic searching and comparison algorithms, word filtering, and accurate word matching required for reliable statistical analysis.
Kseniia (Database & Architecture)	Manages the MariaDB database schema and implements Data Access Objects (DAOs). Ksenia will structure the project and build a comprehensive, professional software architecture for it.
Luara (Frontend Developer & UX)	Designs the user interface mockups in Figma and builds the JavaFX frontend. Luara focuses on creating a responsive and intuitive layout for the multi-text comparison features.
Unna (DevOps & QA)	Configures the Docker infrastructure and builds the Jenkins CI/CD, deployment pipeline. Unna also writes and confirms JUnit tests to validate the accuracy and working of the functions.

5.2 Software, Hardware, and Tools

Category	Details
Hardware	Development will require standard, medium to powerful workstations with a minimum of 8 GB RAM and a modern processor to comfortably run an IDE alongside local Docker containers for the database, Jenkins and CI/CD tools.

Category	Details
Technology Stack	Java/JavaFX (App), MariaDB (Database), Docker/Jenkins (Infra). The application is built using Java and the JavaFX SDK, with Maven handling dependencies and builds.
Project Management & Infrastructure	The team uses Trello for sprint tracking, GitHub for version control, and various tools for project documentation. Docker is used to containerize the MariaDB environment, and Jenkins automates the testing and build pipeline.

5.3 External Resources and Support

Resource	Purpose
iTEXT -PDF parsing library	Handles the extraction of raw text streams from PDF documents. The foundational solution for this part is already completed.
MariaDB	Facilitates direct and secure communication between the Java application and the database.
JUnit 5	Used to execute automated unit testing on the backend logic and integration testing on the JavaFX user interface.

6. Risk Management

6.1 Corrupted or Unsupported File Inputs

Risk: Corrupted or Unsupported File Inputs.

Description: Users upload corrupted, encrypted, or very heavily stylized PDF-files that cause the iText parser to fail or extract unreadable characters, which subsequently breaks the statistical analysis pipeline.

Likelihood: High

Impact: Medium

Mitigation Strategies: Veikka will implement robust exception handling and strict input validation and sanitation before processing begins. If text extraction fails or returns invalid encoding, the application will gracefully abort the operation and display a clear, instructive error message to the user rather than crashing the system.

6.2 Development Bottlenecks

Risk: Development Bottlenecks.

Description: Because tasks are strictly divided by technicality (backend, database, frontend, DevOps), an unexpected absence of a team member or a severe blocker in one domain or a severe blocker in one domain could severely stall the entire 8-week project pipeline.

Likelihood: Low

Impact: High

Mitigation Strategies: The team will hold mandatory weekly Scrum meetings to keep up-to-date and review Trello progress. Unna will ensure all infrastructure, testing, and deployment processes are thoroughly documented so any team member can run builds or restart local services if needed. Team members will be flexible regarding the tasks and be capable of picking up additional tasks as needed.

6.3 Disruptive Feature or Database Changes

Risk: Disruptive Feature or Database Changes

Description: Too-late changes to the projects features the data model for user profiles, saved text statistics, or quotations could break the project stack in surprising and fatal ways.

Likelihood: Low

Impact: High

Mitigation Strategies: Kseniia will finalize the Entity-Relationship (ER) diagram and lock the core data schema during Sprint 2. By abstracting all database interactions through DAO interfaces, any underlying schema adjustments can be handled without requiring massive rewrites of Veikka's analysis engine or Luara's frontend.

7. Testing and Quality Assurance

Testing will be carried out throughout the development of the Text Analysis application to ensure that the implemented functionality works correctly and that changes to the application do not introduce new errors. The main testing approaches will be unit testing and integration testing implemented using JUnit 5.

Unit testing will be used to test individual components and methods separately. Particular attention will be given to the core text analysis functionality, such as word and phrase searching, handling different forms of words, counting word occurrences, filtering stop words, calculating word frequencies, and comparing statistics between texts. Tests will verify that these functions produce the expected results for different input texts and edge cases.

Integration testing will be used to verify that different parts of the application work correctly together. For example, tests will cover workflows such as uploading and opening a text, processing the text and displaying its analysis results, searching for a word and displaying its context and location, comparing multiple texts, and saving and retrieving analysis results.

The success criteria for testing are that the implemented functionality produces the expected results, the main user stories can be completed successfully, and no critical defects remain before the final release. Tests will also be updated when new functionality is implemented or existing functionality is changed. Test results and identified issues will be monitored during development and addressed through the team's development and Scrum process.

8. Documentation and Reporting

The documentation will be maintained throughout the development of the Text Analysis application. The user stories and application requirements have already been defined and are available in Notion and the GitHub main README, respectively. They describe the main functionality expected from the application and will be used as a guide for development and testing.

Other available design documentation includes the use case diagram, which illustrates the main interactions between the user and the application. The next step is to add the use case diagram to the USER-MANUAL.md, where further instructions for using the analysis and text comparison features will also be added.

The remaining design documentation to be completed includes the Figma interface mockup, the UML architecture diagram, which will provide a clear overview of the application's architecture and development pipeline, including the deployment process, and the database ER diagram, which will describe the main data entities, their attributes, and relationships required for storing user profiles, texts, search results, quotations, and analysis results.

The main README.md file will also be used to document the technologies used in the project, as well as the installation and setup instructions required to run the application. In addition, Javadoc will be used to document important classes, interfaces, and methods in the core parts of the application. This documentation will describe their purpose and responsibilities, particularly within the text analysis functionality.

Project progress will be monitored through development sprints and weekly Scrum meetings. During the meetings, the team will review completed work, discuss current progress, identify problems or blockers, and plan the development tasks for the following week. Trello will be used to organize and track project tasks. Tasks will be updated according to their status and the discussions held during Scrum meetings. This will allow the team to monitor completed, ongoing, and planned work for each week, while also serving as a guide for the Scrum meetings.